



Threagile

Agile Threat Modeling

Threat Model Report

Greeting Application

1 January 2026

Alice Example

Table of Contents

Results Overview

Management Summary	4
Impact Analysis of 10 Initial Risks in 9 Categories	5
Risk Mitigation	6
Asset Register	7
Impact Analysis of 8 Remaining Risks in 7 Categories	10
Application Overview	11
Data-Flow Diagram	12
Security Requirements	13
Abuse Cases	14
Tag Listing	15
STRIDE Classification of Identified Risks	16
Assignment by Function	18
RAA Analysis	20
Data Mapping	21
Out-of-Scope Assets: 1 Asset	22
Potential Model Failures: 2 / 2 Risks	23
Questions: 0 / 0 Questions	24

Risks by Vulnerability category

Identified Risks by Vulnerability category	25
Missing Authentication: 1 / 1 Risk	26
Missing Hardening: 1 / 1 Risk	28
Server-Side Request Forgery (SSRF): 1 / 1 Risk	30
Unencrypted Communication: 2 / 2 Risks	32
Missing Build Infrastructure: 1 / 1 Risk	34
Missing Vault (Secret Storage): 1 / 1 Risk	36
Missing Web Application Firewall (WAF): 1 / 1 Risk	38
Missing Two-Factor Authentication (2FA): 0 / 1 Risk	40
Untrusted Deserialization: 0 / 1 Risk	42

Risks by Technical Asset

Identified Risks by Technical Asset	44
Python HTTP Service: 7 / 9 Risks	45
Public Clients: out-of-scope	49

Data Breach Probabilities by Data Asset

Identified Data Breach Probabilities by Data Asset	52
JSON Request: 6 / 8 Risks	53
JSON Response: 6 / 8 Risks	54

Trust Boundaries

Corporate Edge	55
Public Internet	55

About Threagile

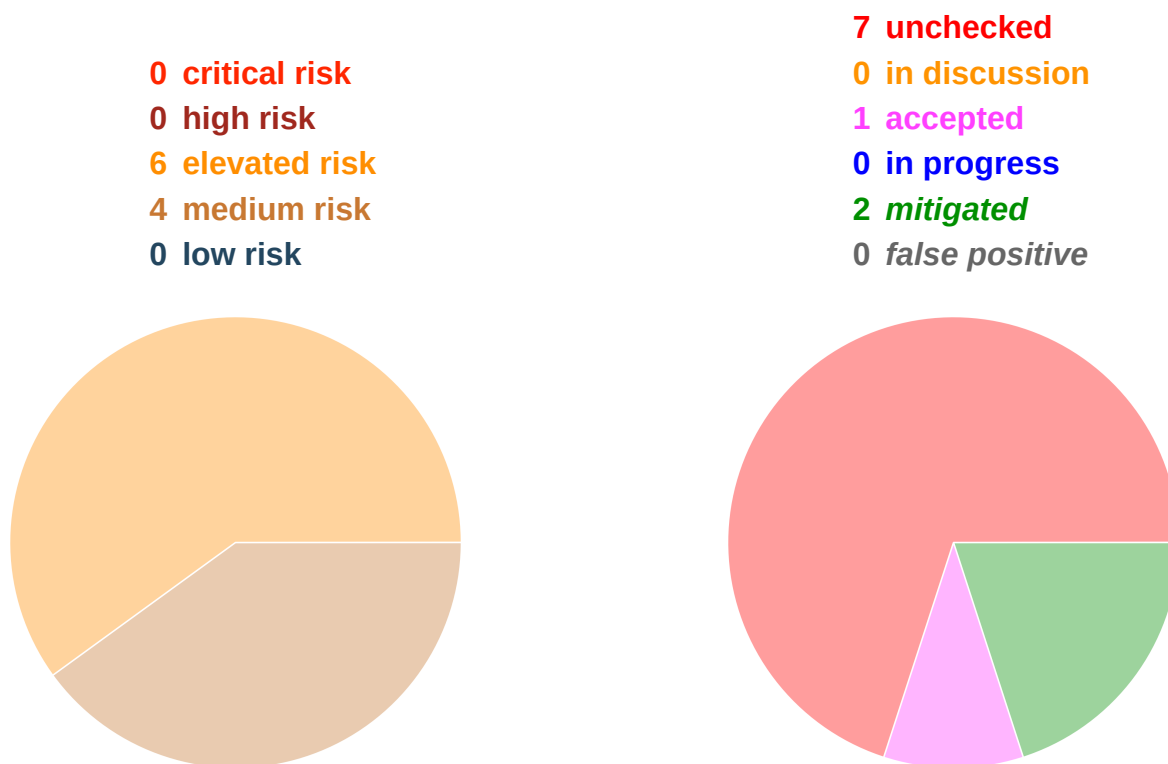
Risk Rules Checked by Threagile	57
Disclaimer	58

Management Summary

Threagile toolkit was used to model the architecture of "Greeting Application" and derive risks by analyzing the components and data flows. The risks identified during this analysis are shown in the following chapters. Identified risks during threat modeling do not necessarily mean that the vulnerability associated with this risk actually exists: it is more to be seen as a list of potential risks and threats, which should be individually reviewed and reduced by removing false positives. For the remaining risks it should be checked in the design and implementation of "Greeting Application" whether the mitigation advices have been applied or not.

Each risk finding references a chapter of the OWASP ASVS (Application Security Verification Standard) audit checklist. The OWASP ASVS checklist should be considered as an inspiration by architects and developers to further harden the application in a Defense-in-Depth approach. Additionally, for each risk finding a link towards a matching OWASP Cheat Sheet or similar with technical details about how to implement a mitigation is given.

In total **10 initial risks** in **9 categories** have been identified during the threat modeling process:



Just a sample app to demonstrate threat modeling of an API Endpoint

Impact Analysis of 10 Initial Risks in 9 Categories

The most prevalent impacts of the **10 initial risks** (distributed over **9 risk categories**) are (taking the severity ratings into account and using the highest for each category):

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated: **Missing Authentication**: 1 Initial Risk - Exploitation likelihood is *Likely with Medium impact*.

If this risk is unmitigated, attackers might be able to access or modify sensitive data in an unauthenticated way.

Elevated: **Missing Hardening**: 1 Initial Risk - Exploitation likelihood is *Likely with Medium impact*.
If this risk remains unmitigated, attackers might be able to easier attack high-value targets.

Elevated: **Server-Side Request Forgery (SSRF)**: 1 Initial Risk - Exploitation likelihood is *Likely with Medium impact*.

If this risk is unmitigated, attackers might be able to access sensitive services or files of network-reachable components by modifying outgoing calls of affected components.

Elevated: **Unencrypted Communication**: 2 Initial Risks - Exploitation likelihood is *Likely with Medium impact*.

If this risk is unmitigated, network attackers might be able to eavesdrop on unencrypted sensitive data sent between components.

Elevated: **Untrusted Deserialization**: 1 Initial Risk - Exploitation likelihood is *Likely with Very High impact*.

If this risk is unmitigated, attackers might be able to execute code on target systems by exploiting untrusted deserialization endpoints.

Medium: **Missing Build Infrastructure**: 1 Initial Risk - Exploitation likelihood is *Unlikely with Medium impact*.

If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model due to critical build infrastructure components missing in the model.

Medium: **Missing Two-Factor Authentication (2FA)**: 1 Initial Risk - Exploitation likelihood is *Unlikely with Medium impact*.

If this risk is unmitigated, attackers might be able to access or modify highly sensitive data without strong authentication.

Medium: **Missing Vault (Secret Storage)**: 1 Initial Risk - Exploitation likelihood is *Unlikely with Medium impact*.

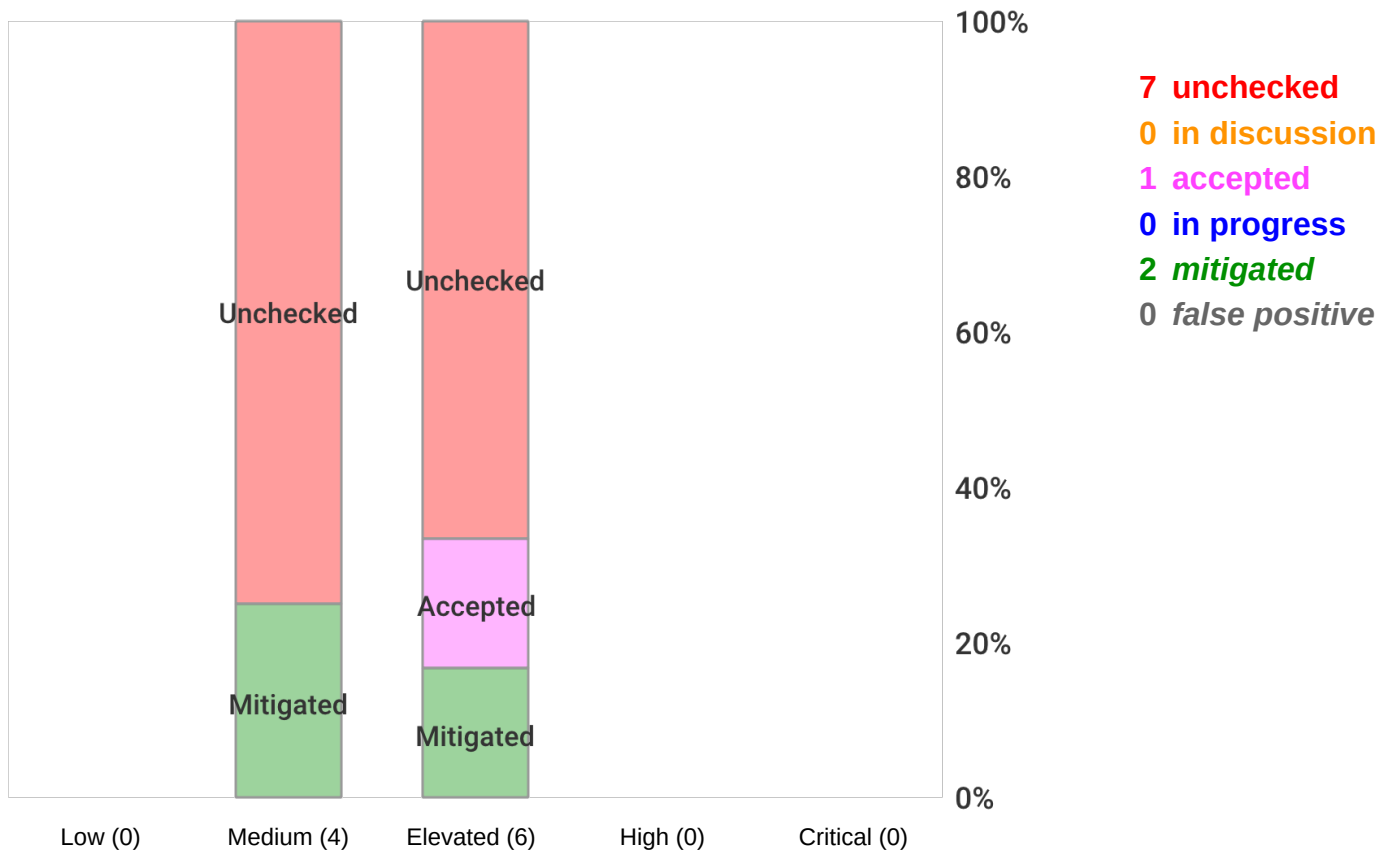
If this risk is unmitigated, attackers might be able to easier steal config secrets (like credentials, private keys, client certificates, etc.) once a vulnerability to access files is present and exploited.

Medium: **Missing Web Application Firewall (WAF)**: 1 Initial Risk - Exploitation likelihood is *Unlikely with Medium impact*.

If this risk is unmitigated, attackers might be able to apply standard attack pattern tests at great speed without any filtering.

Risk Mitigation

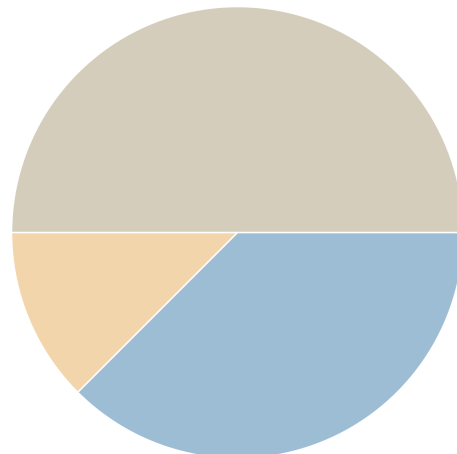
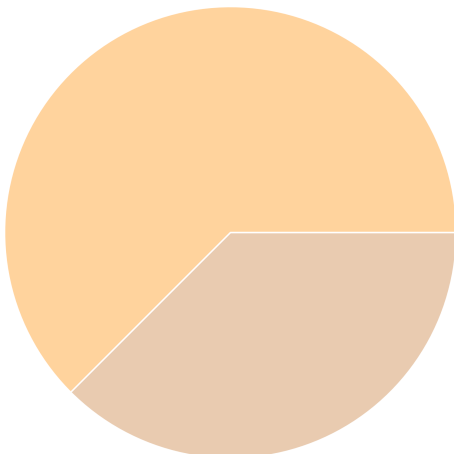
The following chart gives a high-level overview of the risk tracking status (including mitigated risks):



After removal of risks with status *mitigated* and *false positive* the following 8 remain unmitigated:

0 unmitigated critical risk
0 unmitigated high risk
5 unmitigated elevated risk
3 unmitigated medium risk
0 unmitigated low risk

0 business side related
3 architecture related
1 development related
4 operations related



Asset Register

Technical Assets

Public Clients: out-of-scope

Anonymous client on the public internet sending requests to the API.

Python HTTP Service

The python HTTP server that implements the /helloWorld endpoint

Data Assets

JSON Request

Payload for the request/response exchange

JSON Response

Payload for the request/response exchange

Impact Analysis of 8 Remaining Risks in 7 Categories

The most prevalent impacts of the **8 remaining risks** (distributed over **7 risk categories**) are (taking the severity ratings into account and using the highest for each category):

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated: Missing Authentication: 1 Remaining Risk - Exploitation likelihood is *Likely with Medium impact*.

If this risk is unmitigated, attackers might be able to access or modify sensitive data in an unauthenticated way.

Elevated: Missing Hardening: 1 Remaining Risk - Exploitation likelihood is *Likely with Medium impact*.

If this risk remains unmitigated, attackers might be able to easier attack high-value targets.

Elevated: Server-Side Request Forgery (SSRF): 1 Remaining Risk - Exploitation likelihood is *Likely with Medium impact*.

If this risk is unmitigated, attackers might be able to access sensitive services or files of network-reachable components by modifying outgoing calls of affected components.

Elevated: Unencrypted Communication: 2 Remaining Risks - Exploitation likelihood is *Likely with Medium impact*.

If this risk is unmitigated, network attackers might be able to to eavesdrop on unencrypted sensitive data sent between components.

Medium: Missing Build Infrastructure: 1 Remaining Risk - Exploitation likelihood is *Unlikely with Medium impact*.

If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model due to critical build infrastructure components missing in the model.

Medium: Missing Vault (Secret Storage): 1 Remaining Risk - Exploitation likelihood is *Unlikely with Medium impact*.

If this risk is unmitigated, attackers might be able to easier steal config secrets (like credentials, private keys, client certificates, etc.) once a vulnerability to access files is present and exploited.

Medium: Missing Web Application Firewall (WAF): 1 Remaining Risk - Exploitation likelihood is *Unlikely with Medium impact*.

If this risk is unmitigated, attackers might be able to apply standard attack pattern tests at great speed without any filtering.

Application Overview

Business Criticality

The overall business criticality of "Greeting Application" was rated as:

(archive | operational | important | **CRITICAL** | mission-critical)

Business Overview

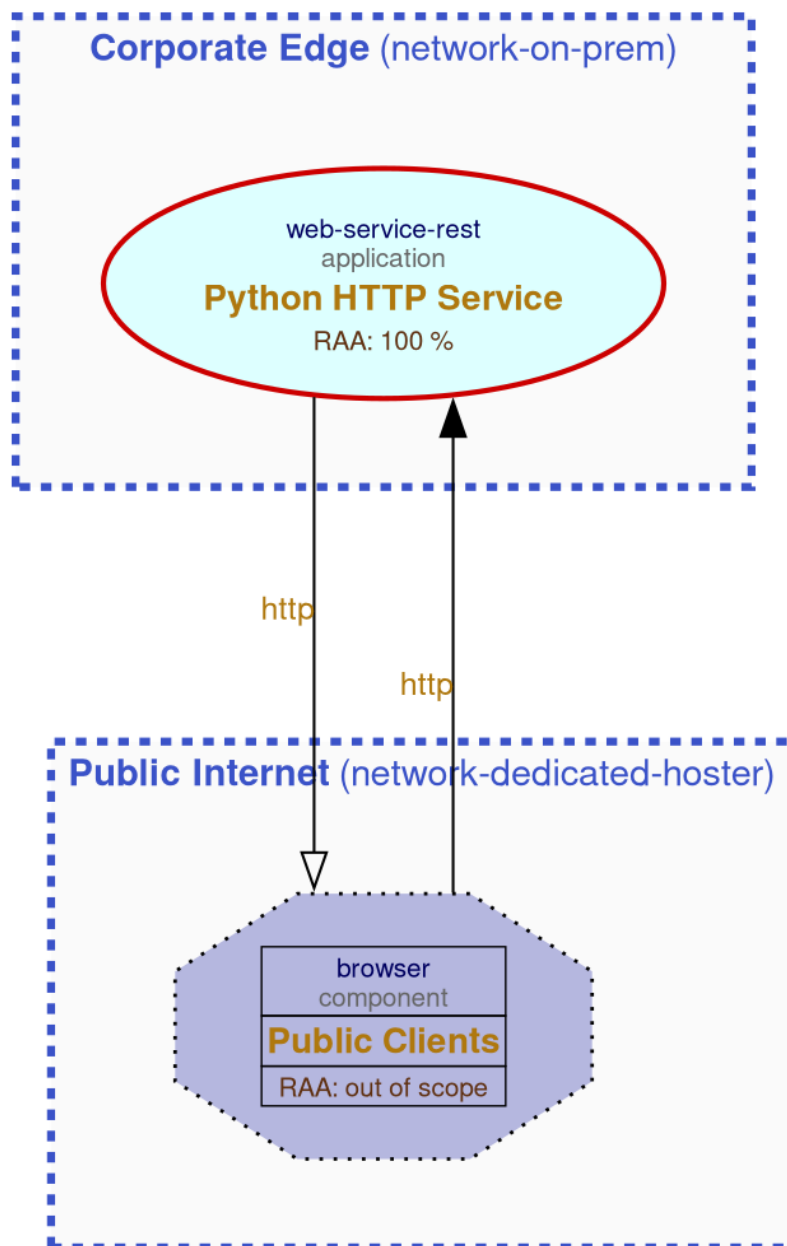
Just an API endpoint for Hello World. Nothing could go wrong at all, could it?

Technical Overview

This API endpoint listens on port 8081 at the route /helloWorld, accepts a json payload with a name, and returns a greeting. It is implemented in Python using the built-in HTTPServer class.

Data-Flow Diagram

The following diagram was generated by Threagile based on the model input and gives a high-level overview of the data-flow between technical assets. The RAA value is the calculated *Relative Attacker Attractiveness* in percent. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.



Security Requirements

This chapter lists the custom security requirements which have been defined for the modeled target.

PHI/PII

Usage of PHI/PII is not allowed in this application. The application should not be used to process any sensitive data.

This list is not complete and regulatory or law relevant security requirements have to be taken into account as well. Also custom individual security requirements might exist for the project.

Abuse Cases

This chapter lists the custom abuse cases which have been defined for the modeled target.

This list is not complete and regulatory or law relevant abuse cases have to be taken into account as well. Also custom individual abuse cases might exist for the project.

Tag Listing

This chapter lists what tags are used by which elements.

apiendpoint

Python HTTP Service

edge

Corporate Edge

httpserver

Python HTTP Service

jsonpayload

JSON Request, JSON Response

python

Python HTTP Service

STRIDE Classification of Identified Risks

This chapter clusters and classifies the risks by STRIDE categories: In total **10 potential risks** have been identified during the threat modeling process of which **0 in the Spoofing** category, **4 in the Tampering** category, **0 in the Repudiation** category, **4 in the Information Disclosure** category, **0 in the Denial of Service** category, and **2 in the Elevation of Privilege** category.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Spoofing

n/a

Tampering

Elevated: Missing Hardening: 1 / 1 Risk - Exploitation likelihood is *Likely* with *Medium* impact.

Technical assets with a Relative Attacker Attractiveness (RAA) value of 55 % or higher should be explicitly hardened taking best practices and vendor hardening guides into account.

Elevated: Untrusted Deserialization: 0 / 1 Risk - Exploitation likelihood is *Likely* with *Very High* impact.

When a technical asset accepts data in a specific serialized form (like Java or .NET serialization), Untrusted Deserialization risks might arise.

Medium: Missing Build Infrastructure: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

The modeled architecture does not contain a build infrastructure (devops-client, sourcecode-repo, build-pipeline, etc.), which might be the risk of a model missing critical assets (and thus not seeing their risks). If the architecture contains custom-developed parts, the pipeline where code gets developed and built needs to be part of the model.

Medium: Missing Web Application Firewall (WAF): 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

To have a first line of filtering defense, security architectures with web-services or web-applications should include a WAF in front of them. Even though a WAF is not a replacement for security (all components must be secure even without a WAF) it adds another layer of defense to the overall system by delaying some attacks and having easier attack alerting through it.

Repudiation

n/a

Information Disclosure

Elevated: Server-Side Request Forgery (SSRF): 1 / 1 Risk - Exploitation likelihood is *Likely* with *Medium* impact.

When a server system (i.e. not a client) is accessing other server systems via typical web protocols Server-Side Request Forgery (SSRF) or Local-File-Inclusion (LFI) or Remote-File-Inclusion (RFI) risks might arise.

Elevated: Unencrypted Communication: 2 / 2 Risks - Exploitation likelihood is *Likely* with *Medium* impact.

Due to the confidentiality and/or integrity rating of the data assets transferred over the communication link this connection must be encrypted.

Medium: Missing Vault (Secret Storage): 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication, authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.

Denial of Service

n/a

Elevation of Privilege

Elevated: Missing Authentication: 1 / 1 Risk - Exploitation likelihood is *Likely* with *Medium* impact.

Technical assets (especially multi-tenant systems) should authenticate incoming requests when the asset processes sensitive data.

Medium: Missing Two-Factor Authentication (2FA): 0 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

Technical assets (especially multi-tenant systems) should authenticate incoming requests with two-factor (2FA) authentication when the asset processes or stores highly sensitive data (in terms of confidentiality, integrity, and availability) and is accessed by humans.

Assignment by Function

This chapter clusters and assigns the risks by functions which are most likely able to check and mitigate them: In total **10 potential risks** have been identified during the threat modeling process of which **1 should be checked by Business Side**, **4 should be checked by Architecture**, **1 should be checked by Development**, and **4 should be checked by Operations**.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Business Side

Medium: **Missing Two-Factor Authentication (2FA)**: 0 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

Apply an authentication method to the technical asset protecting highly sensitive data via two-factor authentication for human users.

Architecture

Elevated: **Missing Authentication**: 1 / 1 Risk - Exploitation likelihood is *Likely* with *Medium* impact.

Apply an authentication method to the technical asset. To protect highly sensitive data consider the use of two-factor authentication for human users.

Elevated: **Untrusted Deserialization**: 0 / 1 Risk - Exploitation likelihood is *Likely* with *Very High* impact.

Try to avoid the deserialization of untrusted data (even of data within the same trust-boundary as long as it is sent across a remote connection) in order to stay safe from Untrusted Deserialization vulnerabilities. Alternatively a strict whitelisting approach of the classes/types/values to deserialize might help as well. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

Medium: **Missing Build Infrastructure**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

Include the build infrastructure in the model.

Medium: **Missing Vault (Secret Storage)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

Consider using a Vault (Secret Storage) to securely store and access config secrets (like credentials, private keys, client certificates, etc.).

Development

Elevated: **Server-Side Request Forgery (SSRF)**: 1 / 1 Risk - Exploitation likelihood is *Likely* with *Medium* impact.

Try to avoid constructing the outgoing target URL with caller controllable values. Alternatively use a mapping (whitelist) when accessing outgoing URLs instead of creating them including caller

controllable values. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

Operations

Elevated: **Missing Hardening**: 1 / 1 Risk - Exploitation likelihood is *Likely* with *Medium* impact.

Try to apply all hardening best practices (like CIS benchmarks, OWASP recommendations, vendor recommendations, DevSec Hardening Framework, DBSAT for Oracle databases, and others).

Elevated: **Unencrypted Communication**: 2 / 2 Risks - Exploitation likelihood is *Likely* with *Medium* impact.

Apply transport layer encryption to the communication link.

Medium: **Missing Web Application Firewall (WAF)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

Consider placing a Web Application Firewall (WAF) in front of the web-services and/or web-applications. For cloud environments many cloud providers offer pre-configured WAFs. Even reverse proxies can be enhanced by a WAF component via ModSecurity plugins.

RAA Analysis

For each technical asset the **"Relative Attacker Attractiveness"** (RAA) value was calculated in percent. The higher the RAA, the more interesting it is for an attacker to compromise the asset. The calculation algorithm takes the sensitivity ratings and quantities of stored and processed data into account as well as the communication links of the technical asset. Neighbouring assets to high-value RAA targets might receive an increase in their RAA value when they have a communication link towards that target ("Pivoting-Factor").

The following lists all technical assets sorted by their RAA value from highest (most attacker attractive) to lowest. This list can be used to prioritize on efforts relevant for the most attacker-attractive technical assets:

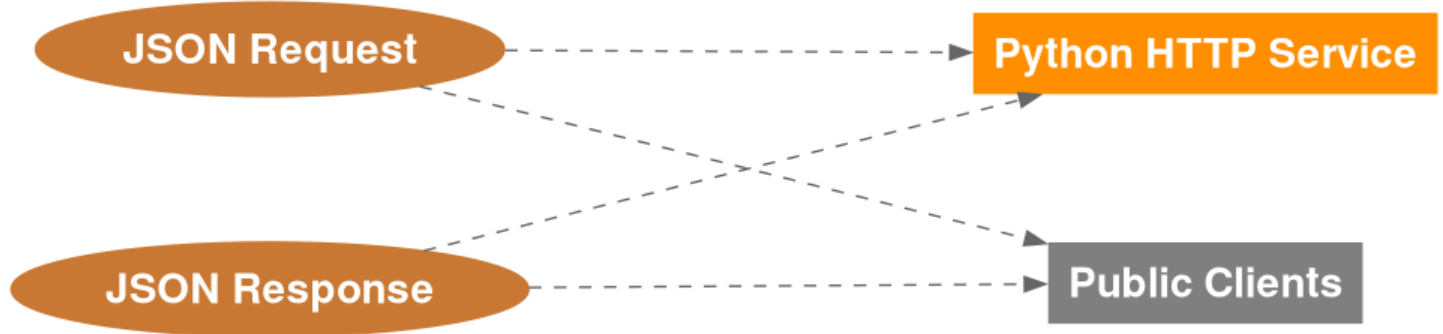
Technical asset paragraphs are clickable and link to the corresponding chapter.

Python HTTP Service: RAA 100%

The python HTTP server that implements the /helloWorld endpoint

Data Mapping

The following diagram was generated by Threagile based on the model input and gives a high-level distribution of data assets across technical assets. The color matches the identified data breach probability and risk level (see the "Data Breach Probabilities" chapter for more details). A solid line stands for *data is stored by the asset* and a dashed one means *data is processed by the asset*. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.



Out-of-Scope Assets: 1 Asset

This chapter lists all technical assets that have been defined as out-of-scope. Each one should be checked in the model whether it should better be included in the overall risk analysis:

Technical asset paragraphs are clickable and link to the corresponding chapter.

Public Clients: out-of-scope

Client devices and browsers are not managed by this application.

Potential Model Failures: 2 / 2 Risks

This chapter lists potential model failures where not all relevant assets have been modeled or the model might itself contain inconsistencies. Each potential model failure should be checked in the model against the architecture design:

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium: Missing Build Infrastructure: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

The modeled architecture does not contain a build infrastructure (devops-client, sourcecode-repo, build-pipeline, etc.), which might be the risk of a model missing critical assets (and thus not seeing their risks). If the architecture contains custom-developed parts, the pipeline where code gets developed and built needs to be part of the model.

Medium: Missing Vault (Secret Storage): 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication, authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.

Questions: 0 / 0 Questions

This chapter lists custom questions that arose during the threat modeling process.

No custom questions arose during the threat modeling process.

Identified Risks by Vulnerability category

In total **10 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 6 as elevated, 4 as medium, and 0 as low.**

These risks are distributed across **9 vulnerability categories**. The following sub-chapters of this section describe each identified risk category.

Missing Authentication: 1 / 1 Risk

Description (Elevation of Privilege): [CWE 306](#)

Technical assets (especially multi-tenant systems) should authenticate incoming requests when the asset processes sensitive data.

Impact

If this risk is unmitigated, attackers might be able to access or modify sensitive data in an unauthenticated way.

Detection Logic

In-scope technical assets (except load-balancer, reverse-proxy, service-registry, waf, ids, and ips and in-process calls) should authenticate incoming requests when the asset processes sensitive data. This is especially the case for all multi-tenant assets (there even non-sensitive ones).

Risk Rating

The risk rating (medium or high) depends on the sensitivity of the data sent across the communication link. Monitoring callers are exempted from this risk.

False Positives

Technical assets which do not process requests regarding functionality or data linked to end-users (customers) can be considered as false positives after individual review.

Mitigation (Architecture): Authentication of Incoming Requests

Apply an authentication method to the technical asset. To protect highly sensitive data consider the use of two-factor authentication for human users.

ASVS Chapter: [V2 - Authentication Verification Requirements](#)

Cheat Sheet: [Authentication_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Authentication** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Authentication covering communication link **Public Greeting API Request from Public Clients to Python HTTP Service**: Exploitation likelihood is *Likely* with *Medium* impact.

[missing-authentication@public-clients>public-greeting-api-request@public-clients@python-http-service](#)

Accepted	2026-01-02	Alice	N/A
----------	------------	-------	-----

This is a public API that is meant to be accessed by anyone, so we accept the risk of missing authentication.

Missing Hardening: 1 / 1 Risk

Description (Tampering): [CWE 16](#)

Technical assets with a Relative Attacker Attractiveness (RAA) value of 55 % or higher should be explicitly hardened taking best practices and vendor hardening guides into account.

Impact

If this risk remains unmitigated, attackers might be able to easier attack high-value targets.

Detection Logic

In-scope technical assets with RAA values of 55 % or higher. Generally for high-value targets like data stores, application servers, identity providers and ERP systems this limit is reduced to 40 %

Risk Rating

The risk rating depends on the sensitivity of the data processed in the technical asset.

False Positives

Usually no false positives.

Mitigation (Operations): System Hardening

Try to apply all hardening best practices (like CIS benchmarks, OWASP recommendations, vendor recommendations, DevSec Hardening Framework, DBSAT for Oracle databases, and others).

ASVS Chapter: [V14 - Configuration Verification Requirements](#)

Cheat Sheet: [Attack Surface Analysis Cheat Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Hardening** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Hardening risk at **Python HTTP Service**: Exploitation likelihood is *Likely* with *Medium* impact.

[missing-hardening@python-http-service](#)

Unchecked

Server-Side Request Forgery (SSRF): 1 / 1 Risk

Description (Information Disclosure): [CWE 918](#)

When a server system (i.e. not a client) is accessing other server systems via typical web protocols Server-Side Request Forgery (SSRF) or Local-File-Inclusion (LFI) or Remote-File-Inclusion (RFI) risks might arise.

Impact

If this risk is unmitigated, attackers might be able to access sensitive services or files of network-reachable components by modifying outgoing calls of affected components.

Detection Logic

In-scope non-client systems accessing (using outgoing communication links) targets with either HTTP or HTTPS protocol.

Risk Rating

The risk rating (low or medium) depends on the sensitivity of the data assets receivable via web protocols from targets within the same network trust-boundary as well on the sensitivity of the data assets receivable via web protocols from the target asset itself. Also for cloud-based environments the exploitation impact is at least medium, as cloud backend services can be attacked via SSRF.

False Positives

Servers not sending outgoing web requests can be considered as false positives after review.

Mitigation (Development): SSRF Prevention

Try to avoid constructing the outgoing target URL with caller controllable values. Alternatively use a mapping (whitelist) when accessing outgoing URLs instead of creating them including caller controllable values. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

ASVS Chapter: [V12 - File and Resources Verification Requirements](#)

Cheat Sheet: [Server_Side_Request_Forgery_Prevention_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Server-Side Request Forgery (SSRF)** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Server-Side Request Forgery (SSRF) risk at **Python HTTP Service** server-side web-requesting the target **Public Clients** via **Public Greeting API Response**: Exploitation likelihood is *Likely with Medium* impact.

server-side-request-forgery@python-http-service@public-clients@python-http-service>public-greeting-api-response

Unchecked

Unencrypted Communication: 2 / 2 Risks

Description (Information Disclosure): [CWE 319](#)

Due to the confidentiality and/or integrity rating of the data assets transferred over the communication link this connection must be encrypted.

Impact

If this risk is unmitigated, network attackers might be able to eavesdrop on unencrypted sensitive data sent between components.

Detection Logic

Unencrypted technical communication links of in-scope technical assets (excluding monitoring traffic as well as local-file-access, in-process-library-call and inter-process-communication) transferring sensitive data.

Risk Rating

Depending on the confidentiality rating of the transferred data-assets either medium or high risk.

False Positives

When all sensitive data sent over the communication link is already fully encrypted on document or data level. Also intra-container/pod communication can be considered false positive when container orchestration platform handles encryption.

Mitigation (Operations): Encryption of Communication Links

Apply transport layer encryption to the communication link.

ASVS Chapter: [V9 - Communication Verification Requirements](#)

Cheat Sheet: [Transport Layer Protection Cheat Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Unencrypted Communication** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Unencrypted Communication named **Public Greeting API Request** between **Public Clients** and **Python HTTP Service**: Exploitation likelihood is *Likely* with *Medium* impact.

[unencrypted-communication@public-clients>public-greeting-api-request@public-clients@python-http-service](#)

Unchecked

Unencrypted Communication named **Public Greeting API Response** between **Python HTTP Service** and **Public Clients**: Exploitation likelihood is *Likely* with *Medium* impact.

[unencrypted-communication@python-http-service>public-greeting-api-response@python-http-service@public-clients](#)

Unchecked

Missing Build Infrastructure: 1 / 1 Risk

Description (Tampering): [CWE 1127](#)

The modeled architecture does not contain a build infrastructure (devops-client, sourcecode-repo, build-pipeline, etc.), which might be the risk of a model missing critical assets (and thus not seeing their risks). If the architecture contains custom-developed parts, the pipeline where code gets developed and built needs to be part of the model.

Impact

If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model due to critical build infrastructure components missing in the model.

Detection Logic

Models with in-scope custom-developed parts missing in-scope development (code creation) and build infrastructure components (devops-client, sourcecode-repo, build-pipeline, etc.).

Risk Rating

The risk rating depends on the highest sensitivity of the in-scope assets running custom-developed parts.

False Positives

Models not having any custom-developed parts can be considered as false positives after individual review.

Mitigation (Architecture): Build Pipeline Hardening

Include the build infrastructure in the model.

ASVS Chapter: [V1 - Architecture, Design and Threat Modeling Requirements](#)

Cheat Sheet: [Attack_Surface_Analysis_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Build Infrastructure** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Build Infrastructure in the threat model (referencing asset **Python HTTP Service** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

[missing-build-infrastructure@python-http-service](#)

Unchecked

Missing Vault (Secret Storage): 1 / 1 Risk

Description (Information Disclosure): [CWE 522](#)

In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication, authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.

Impact

If this risk is unmitigated, attackers might be able to easier steal config secrets (like credentials, private keys, client certificates, etc.) once a vulnerability to access files is present and exploited.

Detection Logic

Models without a Vault (Secret Storage).

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed.

False Positives

Models where no technical assets have any kind of sensitive config data to protect can be considered as false positives after individual review.

Mitigation (Architecture): Vault (Secret Storage)

Consider using a Vault (Secret Storage) to securely store and access config secrets (like credentials, private keys, client certificates, etc.).

ASVS Chapter: [V6 - Stored Cryptography Verification Requirements](#)

Cheat Sheet: [Cryptographic_Storage_Cheat_Sheet](#)

Check

Is there a Vault (Secret Storage) in place?

Risk Findings

The risk **Missing Vault (Secret Storage)** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Vault (Secret Storage) in the threat model (referencing asset **Python HTTP Service** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

[missing-vault@python-http-service](#)

Unchecked

Missing Web Application Firewall (WAF): 1 / 1 Risk

Description (Tampering): [CWE 1008](#)

To have a first line of filtering defense, security architectures with web-services or web-applications should include a WAF in front of them. Even though a WAF is not a replacement for security (all components must be secure even without a WAF) it adds another layer of defense to the overall system by delaying some attacks and having easier attack alerting through it.

Impact

If this risk is unmitigated, attackers might be able to apply standard attack pattern tests at great speed without any filtering.

Detection Logic

In-scope web-services and/or web-applications accessed across a network trust boundary not having a Web Application Firewall (WAF) in front of them.

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed.

False Positives

Targets only accessible via WAFs or reverse proxies containing a WAF component (like ModSecurity) can be considered as false positives after individual review.

Mitigation (Operations): Web Application Firewall (WAF)

Consider placing a Web Application Firewall (WAF) in front of the web-services and/or web-applications. For cloud environments many cloud providers offer pre-configured WAFs. Even reverse proxies can be enhanced by a WAF component via ModSecurity plugins.

ASVS Chapter: [V1 - Architecture, Design and Threat Modeling Requirements](#)

Cheat Sheet: [Virtual Patching Cheat Sheet](#)

Check

Is there a Web Application Firewall (WAF) in place?

Risk Findings

The risk **Missing Web Application Firewall (WAF)** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Web Application Firewall (WAF) risk at **Python HTTP Service**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[missing-waf@python-http-service](#)

Unchecked

Missing Two-Factor Authentication (2FA): 0 / 1 Risk

Description (Elevation of Privilege): [CWE 308](#)

Technical assets (especially multi-tenant systems) should authenticate incoming requests with two-factor (2FA) authentication when the asset processes or stores highly sensitive data (in terms of confidentiality, integrity, and availability) and is accessed by humans.

Impact

If this risk is unmitigated, attackers might be able to access or modify highly sensitive data without strong authentication.

Detection Logic

In-scope technical assets (except load-balancer, reverse-proxy, waf, ids, and ips) should authenticate incoming requests via two-factor authentication (2FA) when the asset processes or stores highly sensitive data (in terms of confidentiality, integrity, and availability) and is accessed by a client used by a human user.

Risk Rating

medium

False Positives

Technical assets which do not process requests regarding functionality or data linked to end-users (customers) can be considered as false positives after individual review.

Mitigation (Business Side): Authentication with Second Factor (2FA)

Apply an authentication method to the technical asset protecting highly sensitive data via two-factor authentication for human users.

ASVS Chapter: [V2 - Authentication Verification Requirements](#)

Cheat Sheet: [Multifactor_Authentication_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Two-Factor Authentication (2FA)** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Two-Factor Authentication covering communication link **Public Greeting API Request** from **Public Clients** to **Python HTTP Service**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[missing-authentication-second-factor@public-clients>public-greeting-api-request@public-clients@python-http-service](#)

Mitigated	2026-01-02	Alice	N/A
We don't allow authentication at all, so no MFA is required.			

Untrusted Deserialization: 0 / 1 Risk

Description (Tampering): [CWE 502](#)

When a technical asset accepts data in a specific serialized form (like Java or .NET serialization), Untrusted Deserialization risks might arise.

See <https://christian-schneider.net/JavaDeserializationSecurityFAQ.html> for more details.

Impact

If this risk is unmitigated, attackers might be able to execute code on target systems by exploiting untrusted deserialization endpoints.

Detection Logic

In-scope technical assets accepting serialization data formats (including EJB and RMI protocols).

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed.

False Positives

Fully trusted (i.e. cryptographically signed or similar) data deserialized can be considered as false positives after individual review.

Mitigation (Architecture): Prevention of Deserialization of Untrusted Data

Try to avoid the deserialization of untrusted data (even of data within the same trust-boundary as long as it is sent across a remote connection) in order to stay safe from Untrusted Deserialization vulnerabilities. Alternatively a strict whitelisting approach of the classes/types/values to deserialize might help as well. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

ASVS Chapter: [V5 - Validation, Sanitization and Encoding Verification Requirements](#)

Cheat Sheet: [Deserialization_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Untrusted Deserialization** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.
Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Untrusted Deserialization risk at **Python HTTP Service**: Exploitation likelihood is *Likely* with *Very High* impact.

[untrusted-deserialization@python-http-service](#)

Mitigated	2026-01-02	Alice	N/A
-----------	------------	-------	-----

The API endpoint will use a safe deserialization library that prevents code execution and other attacks.

Identified Risks by Technical Asset

In total **10 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 6 as elevated, 4 as medium, and 0 as low.**

These risks are distributed across **1 in-scope technical assets**. The following sub-chapters of this section describe each identified risk grouped by technical asset. The RAA value of a technical asset is the calculated "Relative Attacker Attractiveness" value in percent.

Python HTTP Service: 7 / 9 Risks

Description

The python HTTP server that implements the /helloWorld endpoint

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Hardening risk at **Python HTTP Service**: Exploitation likelihood is *Likely* with *Medium* impact.

missing-hardening@python-http-service

Unchecked

Server-Side Request Forgery (SSRF) risk at **Python HTTP Service** server-side web-requesting the target **Public Clients** via **Public Greeting API Response**: Exploitation likelihood is *Likely* with *Medium* impact.

server-side-request-forgery@python-http-service@public-clients@python-http-service>public-greeting-api-response

Unchecked

Unencrypted Communication named **Public Greeting API Response** between **Python HTTP Service** and **Public Clients**: Exploitation likelihood is *Likely* with *Medium* impact.

unencrypted-communication@python-http-service>public-greeting-api-response@python-http-service@public-clients

Unchecked

Missing Authentication covering communication link **Public Greeting API Request** from **Public Clients** to **Python HTTP Service**: Exploitation likelihood is *Likely* with *Medium* impact.

missing-authentication@public-clients>public-greeting-api-request@public-clients@python-http-service

Accepted

2026-01-02 Alice

N/A

This is a public API that is meant to be accessed by anyone, so we accept the risk of missing authentication.

Untrusted Deserialization risk at **Python HTTP Service**: Exploitation likelihood is *Likely* with *Very High* impact.

untrusted-deserialization@python-http-service

Mitigated

2026-01-02 Alice

N/A

The API endpoint will use a safe deserialization library that prevents code execution and other attacks.

Medium Risk Severity

Missing Build Infrastructure in the threat model (referencing asset **Python HTTP Service** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-build-infrastructure@python-http-service

Unchecked

Missing Vault (Secret Storage) in the threat model (referencing asset **Python HTTP Service** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-vault@python-http-service

Unchecked

Missing Web Application Firewall (WAF) risk at **Python HTTP Service**: Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-waf@python-http-service

Unchecked

Missing Two-Factor Authentication covering communication link **Public Greeting API Request** from **Public Clients** to **Python HTTP Service**: Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-authentication-second-factor@public-clients>public-greeting-api-request@public-clients@python-http-service

Mitigated 2026-01-02 Alice N/A
We don't allow authentication at all, so no MFA is required.

Asset Information

ID:	python-http-service
Type:	process
Usage:	business
RAA:	100 %
Size:	application
Technology:	web-service-rest
Tags:	apiendpoint, httpserver, python
Internet:	true
Machine:	virtual
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	false
Data Processed:	JSON Request, JSON Response
Data Stored:	none
Formats Accepted:	JSON, Serialization

Asset Rating

Owner:	Alice's Development Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)

Availability: operational (rated 2 in scale of 5)
CIA-Justification: We said it's just a hello world API, so no need for any special CIA ratings here. At the same time, the service runs on our server and nobody should be able to access the server without authorization

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Public Greeting API Response (outgoing)

HTTP response from /helloWorld

Target:	Public Clients
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	true
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	JSON Response
Data Received:	JSON Request

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

Public Greeting API Request (incoming)

HTTP request/response sessions to /helloWorld

Source:	Public Clients
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false

IP-Filtered: false
Data Received: JSON Request
Data Sent: JSON Response

Public Clients: out-of-scope

Description

Anonymous client on the public internet sending requests to the API.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Unencrypted Communication named **Public Greeting API Request** between **Public Clients** and **Python HTTP Service**: Exploitation likelihood is *Likely* with *Medium* impact.

[unencrypted-communication@public-clients>public-greeting-api-request@public-clients@python-http-service](#)

Unchecked

Asset Information

ID:	public-clients
Type:	external-entity
Usage:	business
RAA:	out-of-scope
Size:	component
Technology:	browser
Tags:	none
Internet:	false
Machine:	physical
Encryption:	none
Multi-Tenant:	true
Redundant:	false
Custom-Developed:	false
Client by Human:	true
Data Processed:	JSON Request, JSON Response
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	External Untrusted Users	
Confidentiality:	public	(rated 1 in scale of 5)

Integrity: critical (rated 4 in scale of 5)
Availability: operational (rated 2 in scale of 5)
CIA-Justification:

Asset Out-of-Scope Justification

Client devices and browsers are not managed by this application.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Public Greeting API Request (outgoing)

HTTP request/response sessions to /helloWorld

Target:	Python HTTP Service
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	JSON Request
Data Received:	JSON Response

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

Public Greeting API Response (incoming)

HTTP response from /helloWorld

Source:	Python HTTP Service
Protocol:	http
Encrypted:	false
Authentication:	none

Authorization: none
Read-Only: true
Usage: business
Tags: none
VPN: false
IP-Filtered: false
Data Received: JSON Response
Data Sent: JSON Request

Identified Data Breach Probabilities by Data Asset

In total **10 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 0 as high, 6 as elevated, 4 as medium, and 0 as low.**

These risks are distributed across **2 data assets**. The following sub-chapters of this section describe the derived data breach probabilities grouped by data asset.

Technical asset names and risk IDs are clickable and link to the corresponding chapter.

JSON Request: 6 / 8 Risks

Payload for the request/response exchange

ID:	json-request	
Usage:	business	
Quantity:	very-few	
Tags:	jsonpayload	
Origin:	Untrusted Clients	
Owner:	Alice's Development Team	
Confidentiality:	public	(rated 1 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	We're just doing hello world, so no need for any special CIA ratings here.	
Processed by:	Public Clients, Python HTTP Service	
Stored by:	none	
Sent via:	Public Greeting API Request	
Received via:	Public Greeting API Response	
Data Breach:	possible	
Data Breach Risks:	This data asset has data breach potential because of 6 remaining risksStr:	
	Possible: server-side-request-forgery@python-http-service@public-clients@python-http-service>public-greeting-api-response	
	Possible: unencrypted-communication@public-clients>public-greeting-api-request@public-clients@python-http-service	
	Possible: unencrypted-communication@python-http-service>public-greeting-api-response@python-http-service@public-clients	
	Possible: missing-authentication@public-clients>public-greeting-api-request@public-clients@python-http-service	
	Improbable: missing-hardening@python-http-service	
	Improbable: missing-waf@python-http-service	

JSON Response: 6 / 8 Risks

Payload for the request/response exchange

ID:	json-response	
Usage:	business	
Quantity:	very-few	
Tags:	jsonpayload	
Origin:	python-http-server	
Owner:		
Confidentiality:	public	(rated 1 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	We're just responding to the request so we might assume nothing we send back to the client is dangerous. Is that a fair assumption?	
Processed by:	Public Clients, Python HTTP Service	
Stored by:	none	
Sent via:	Public Greeting API Response	
Received via:	Public Greeting API Request	
Data Breach:	possible	
Data Breach Risks:	This data asset has data breach potential because of 6 remaining risksStr:	

Possible: server-side-request-forgery@python-http-service@public-clients@python-http-service>public-greeting-api-response

Possible: unencrypted-communication@public-clients>public-greeting-api-request@public-clients@python-http-service

Possible: unencrypted-communication@python-http-service>public-greeting-api-response@python-http-service@public-clients

Possible: missing-authentication@public-clients>public-greeting-api-request@public-clients@python-http-service

Improbable: missing-hardening@python-http-service

Improbable: missing-waf@python-http-service

Trust Boundaries

In total **2 trust boundaries** have been modeled during the threat modeling process.

Corporate Edge

Intranet boundary between the public internet and internal resources

ID: corporate-edge
Type: network-on-prem
Tags: edge
Assets inside: Python HTTP Service
Boundaries nested: none

Public Internet

Untrusted public network from which untrusted consumers access the API.

ID: public-internet
Type: network-dedicated-hoster
Tags: none
Assets inside: Public Clients
Boundaries nested: none

Shared Runtimes

In total **0 shared runtime** has been modeled during the threat modeling process.

Risk Rules Checked by Threagile

Threagile Version: 1.0.0

Threagile Build Timestamp: 20260918123354

Threagile Execution Timestamp: 20260922203941

Model Filename: /app/work/sample-model.yaml

Model Hash (SHA256): 4c7b465acc921c600235ea65cb4b0cca035f3518364355c8aacd548fa531bc08

Threagile (see <https://threagile.io> for more details) is an open-source toolkit for agile threat modeling, created by Christian Schneider (<https://christian-schneider.net>): It allows to model an architecture with its assets in an agile fashion as a YAML file directly inside the IDE. Upon execution of the Threagile toolkit all standard risk rules (as well as individual custom rules if present) are checked against the architecture model. At the time the Threagile toolkit was executed on the model input file the following risk rules were checked:

Disclaimer

Alice Example conducted this threat analysis using the open-source Threagile toolkit on the applications and systems that were modeled as of this report's date. Information security threats are continually changing, with new vulnerabilities discovered on a daily basis, and no application can ever be 100% secure no matter how much threat modeling is conducted. It is recommended to execute threat modeling and also penetration testing on a regular basis (for example yearly) to ensure a high ongoing level of security and constantly check for new attack vectors.

This report cannot and does not protect against personal or business loss as the result of use of the applications or systems described. Alice Example and the Threagile toolkit offers no warranties, representations or legal certifications concerning the applications or systems it tests. All software includes defects: nothing in this document is intended to represent or warrant that threat modeling was complete and without error, nor does this document represent or warrant that the architecture analyzed is suitable to task, free of other defects than reported, fully compliant with any industry standards, or fully compatible with any operating system, hardware, or other application. Threat modeling tries to analyze the modeled architecture without having access to a real working system and thus cannot and does not test the implementation for defects and vulnerabilities. These kinds of checks would only be possible with a separate code review and penetration test against a working system and not via a threat model.

By using the resulting information you agree that Alice Example and the Threagile toolkit shall be held harmless in any event.

This report is confidential and intended for internal, confidential use by the client. The recipient is obligated to ensure the highly confidential contents are kept secret. The recipient assumes responsibility for further distribution of this document.

In this particular project, a time box approach was used to define the analysis effort. This means that the author allotted a prearranged amount of time to identify and document threats. Because of this, there is no guarantee that all possible threats and risks are discovered. Furthermore, the analysis applies to a snapshot of the current state of the modeled architecture (based on the architecture information provided by the customer) at the examination time.

Report Distribution

Distribution of this report (in full or in part like diagrams or risk findings) requires that this disclaimer as well as the chapter about the Threagile toolkit and method used is kept intact as part of the distributed report or referenced from the distributed parts.